Informe del trabajo práctico Delivery Rush

Integrantes del grupo:

Daniel Vanney : 45.752.995 – danielvanney04@gmail.com

Juárez, Agustina Soledad: 40.855.602 – Agustina.soledad.juarez@gmail.com

Índice:

Descripción del proyecto

- Funcionamiento general
- Organización del código
- Clase Juego
- Clase Mapa
- Clase Casa
- Clase Negocio
- Clase Repartidor
- Clase Pedido
- Clase Enemigo
- Clase Bomba
- Clase Proyectil

Problemas encontrados y decisiones tomadas

- Construcción del mapa y uso de arreglos
- Coordenadas y tamaño de los dibujos
- Movimiento y dirección del disparo
- Retiro y entrega de pedidos
- Medición del tiempo y vencimientos
- Daño de las bombas y eliminación de objetos
- Enemigos y condiciones de finalización
- Revisión del funcionamiento

Implementación

- Código de Juego
- Código de Mapa
- Código de Casa
- Código de Negocio
- Código de Repartidor
- Código de Pedido
- Código de Enemigo
- Código de Bomba
- Código de Proyectil

Conclusiones

Introducción

En este trabajo desarrollamos Delivery Rush, un videojuego en Java en el que el jugador controla a un repartidor que debe retirar pedidos de distintos negocios y llevarlos a las casas indicadas. El recorrido se realiza por las calles de una ciudad y presenta dos dificultades principales. Cada pedido tiene un tiempo limitado y durante el trayecto aparecen enemigos que pueden quitarle una vida al repartidor y robarle el encargo que lleva.

Partimos del proyecto base de la materia y utilizamos la biblioteca entorno.jar para mostrar los elementos en pantalla y recibir las acciones del teclado y del mouse. A partir de esa base construimos el mapa, representamos los personajes y edificios mediante objetos e incorporamos las reglas de movimiento, entrega, colisión y finalización de la partida.

El desarrollo permitió trabajar con clases, constructores, variables de instancia, métodos y arreglos de objetos. También fue necesario coordinar acciones que ocurren durante una misma actualización del juego. Por ejemplo, antes de aceptar una entrega hay que comprobar que el pedido siga vigente, y después de eliminar un enemigo hay que generar su reemplazo.

En este informe describimos el funcionamiento del juego y la organización del programa. Explicamos las responsabilidades de cada clase, los datos que conserva y los métodos que utiliza. Luego presentamos los problemas que surgieron durante el desarrollo, las soluciones adoptadas y el código fuente de la versión final.

Descripción del proyecto

Funcionamiento general

Al comenzar la partida se eligen al azar una casa y un negocio para formar el primer pedido. Una flecha amarilla indica el lugar de retiro. El jugador debe ubicar al repartidor sobre esa flecha y hacer clic derecho en el negocio. Una vez retirado el pedido, aparece una flecha roja que señala la casa de destino. Para entregar se repite la misma acción frente a la casa.

Cada pedido dispone de 30 segundos desde que se genera. Ese plazo incluye el tiempo que permanece esperando en el negocio y el recorrido hasta la casa. Si el tiempo se agota, el pedido se pierde y se crea uno nuevo. También se genera otro encargo después de una entrega o de un robo. De esta manera, durante una partida activa se trabaja con un pedido a la vez.

El repartidor se maneja con las teclas W,A,S,D y con los botones izquierdo y derecho del mouse.

En el mapa hay cuatro enemigos que recorren una calle en línea recta y cambian de sentido al alcanzar sus extremos. Un choque con cualquiera de ellos quita una vida. Si el repartidor lleva un pedido, también lo pierde. Al menos uno de los enemigos tiene la capacidad de colocar bombas. Estas explotan dos segundos después de aparecer y mantienen su explosión durante medio segundo. El contacto con una bomba o con su área de explosión termina la partida.

El repartidor puede defenderse con el botón izquierdo del mouse. El disparo sale en la última dirección en la que logró moverse. Solo puede haber un proyectil en vuelo. Cuando alcanza a un enemigo, ambos se eliminan y se crea un nuevo enemigo para conservar la cantidad prevista.

La partida se gana al completar cinco entregas. Se pierde cuando las vidas llegan a cero, cuando se alcanza el límite de cinco pedidos robados o cuando una bomba afecta al repartidor. Con las tres vidas iniciales y sin recuperación de vidas, los choques pueden terminar la partida antes de alcanzar el límite de robos. Ambos controles se conservan por separado en el código.

El panel muestra el tiempo transcurrido, las vidas, las entregas, los enemigos eliminados, el estado del pedido, su plazo restante y la cantidad de pedidos robados.

Organización del código

El programa está organizado en nueve clases dentro del paquete juego. Juego coordina la partida y recibe las acciones del jugador. Mapa organiza la ciudad y determina qué posiciones se pueden ocupar. Casa y Negocio representan los edificios. Repartidor contiene los datos y movimientos del personaje controlado por el usuario. Pedido relaciona un negocio con una casa y registra el estado del encargo. Enemigo, Bomba y Proyectil representan los elementos que intervienen en los encuentros y ataques.

Esta distribución permite que cada objeto conserve la información que le corresponde. Por ejemplo, una bomba sabe cuánto tiempo pasó desde su creación y si todavía puede causar daño. La decisión de terminar la partida cuando esa bomba alcanza al repartidor queda en Juego, que administra las vidas y el estado general.

Las casas, los negocios, los enemigos y las bombas se guardan en arreglos de objetos. El pedido actual y el proyectil se representan mediante una referencia individual. En el caso del proyectil, una referencia con valor null indica que se puede disparar nuevamente. Para los enemigos y las bombas, los lugares que pasan a null representan objetos que ya no están activos.

Clase Juego

Juego es la clase que inicia y coordina la aplicación. Mantiene la estructura Juego extends InterfaceJuego que requiere el proyecto base. El entorno utiliza esa relación para ejecutar el método tick de forma repetida mientras la ventana está abierta. Las demás clases del juego no incorporan relaciones de herencia propias.

La variable entorno permite dibujar y consultar el teclado, el mouse y el reloj. Las referencias mapa, repartidor y pedido representan la ciudad, el personaje y el encargo actual. Los arreglos enemigos y bombas contienen los objetos que participan en la partida, mientras que proyectil guarda el único disparo que puede estar en vuelo.

Los contadores pedidosEntregados, pedidosPerdidos, pedidosRobados y enemigosEliminados registran los resultados de las acciones. pedidosPerdidos incluye los encargos vencidos y los robados. pedidosRobados conserva solamente los que se pierden por un choque mientras el repartidor los transporta. vidas indica cuántas oportunidades quedan. vidasIniciales, pedidosParaGanar y pedidosRobadosParaPerder guardan los valores elegidos para comenzar y finalizar la partida.

Para controlar los tiempos se utilizan tiempoActual, inicioPedido y tiempoLimitePedido. tiempoActual guarda la lectura del reloj del entorno, inicioPedido registra cuándo comenzó el encargo y tiempoLimitePedido establece su duración máxima en milisegundos. Los dos primeros atributos son de tipo long y permiten trabajar con valores enteros de tiempo. estadoJuego guarda “Jugando”, “GANASTE” o” PERDISTE” y determina si la simulación debe continuar.

El constructor Juego crea la ventana, el mapa y el repartidor. También carga la imagen del personaje, asigna las tres vidas iniciales, establece la meta de cinco entregas y configura el plazo de 30000 milisegundos. Inicializa los contadores, deja el proyectil en null y crea un arreglo vacío de bombas. Después genera los cuatro enemigos y el primer pedido, establece el estado Jugando e inicia el entorno.

El método generarEnemigo recibe el índice del arreglo donde debe ubicarse el nuevo objeto. Elige si recorrerá una calle horizontal o vertical y calcula una posición sobre el centro de esa calle. El enemigo del índice cero siempre puede dejar bombas. Para los demás, esa capacidad se determina al azar con una probabilidad de 0,4. Si la posición elegida queda demasiado cerca del repartidor, el método desplaza al enemigo hacia el extremo más alejado de la calle. El nuevo objeto recibe el tiempo actual para comenzar su propio intervalo entre bombas.

crearNuevoPedido elige índices aleatorios dentro de los arreglos de negocios y casas. Con esos objetos construye un Pedido, lo pasa al estado Esperando mediante recibirEnNegocio y registra el momento de su creación en inicioPedido. El estado Realizado existe al construir el objeto y se transforma inmediatamente en Esperando, por lo que no permanece visible en el panel.

tick realiza una actualización de la partida. Primero comprueba si el juego terminó. En ese caso dibuja el mensaje final y sale del método. Si sigue activo, consulta el reloj y revisa si venció el pedido. Luego procesa el movimiento del repartidor, los clics de retiro y entrega, los disparos, el desplazamiento de los enemigos y la evolución de las bombas. Finalmente evalúa la victoria o la derrota y dibuja los elementos que continúan activos.

Dentro de tick se conserva la posición anterior antes de intentar cada movimiento. Si Mapa informa una colisión, esa posición se restaura. La dirección del repartidor cambia únicamente cuando el desplazamiento fue aceptado. Para retirar o entregar se comprueban dos condiciones en conjunto, que el repartidor esté sobre la flecha y que el clic se encuentre dentro del edificio correcto. Un pedido que venció durante esa actualización no puede retirarse ni entregarse con el mismo clic.

agregarBomba recibe una bomba nueva y busca primero un lugar libre o una bomba terminada dentro del arreglo. Si encuentra uno, utiliza esa posición y termina la búsqueda. Si todos los lugares están ocupados por bombas activas, crea un arreglo con una posición adicional, copia las referencias existentes y coloca la bomba al final. Así evita ampliar el arreglo cuando todavía hay un espacio que puede reutilizarse.

dibujarPanel muestra los datos de la partida en la parte superior de la ventana. Convierte el tiempo transcurrido a segundos y calcula cuánto le queda al pedido. El plazo visible se redondea hacia arriba para que no aparezcan cero segundos mientras todavía queda una fracción del último segundo. dibujarPantallaFinal recibe el mensaje de resultado y lo escribe sobre un fondo negro. El método main es el punto de entrada del programa y crea una instancia de Juego.

Clase Mapa

Mapa representa la distribución de la ciudad. Sus atributos x e y indican la esquina superior izquierda del área jugable. ancho y alto determinan sus dimensiones. filas y columnas establecen la cantidad de cuadras, mientras que anchoCalle define la separación entre ellas. Los arreglos casas y negocios guardan los edificios creados en el constructor.

El constructor Mapa recibe la posición y el tamaño del mapa. Establece cuatro filas, cuatro columnas y calles de 30 píxeles. Carga las tres imágenes disponibles para las casas y la imagen del negocio. Después calcula el tamaño de cada cuadra descontando del área total el espacio ocupado por las calles, incluidas las de los bordes.

Para ubicar los edificios se utilizan dos ciclos anidados. El primero recorre las filas y el segundo las columnas. Dentro de cada cuadra se crean tres casas y un negocio, todos de 50 por 30 píxeles. El índice del negocio se obtiene a partir de la fila y la columna. Al multiplicar ese índice por tres se obtiene la primera posición de las casas de esa cuadra. Las imágenes de las casas se eligen al azar entre las tres opciones cargadas.

estaEnCalle comprueba que todo el rectángulo del repartidor permanezca dentro del mapa y que no se superponga con ninguna cuadra. Esto impide que el personaje ingrese en el espacio verde entre los edificios. hayColision utiliza esa comprobación y también recorre las casas y los negocios para detectar superposiciones con sus rectángulos. Devuelve true cuando el movimiento debe bloquearse y false cuando la posición es válida.

obtenerYFlechaCasa calcula la coordenada vertical donde debe aparecer la flecha de una casa. Primero identifica su fila y el centro vertical de la cuadra. Si la casa está en la mitad superior, utiliza la calle de arriba. Si está en la mitad inferior, utiliza la calle de abajo. dibujarFlechaCasaRoja combina un rectángulo y un triángulo rojos para mostrar esa indicación y orientar la punta hacia la casa.

obtenerYFlechaNegocio calcula el centro de la calle que queda debajo de la cuadra del negocio. Esa elección coincide con la ubicación de los negocios en la parte inferior de cada cuadra. dibujarFlechaNegocio utiliza la posición calculada para mostrar una flecha amarilla que apunta hacia el edificio.

El método dibujar pinta primero el fondo gris que representa las calles. Luego dibuja las cuadras verdes mediante dos ciclos y recorre los arreglos para pedirle a cada casa y negocio que se dibuje. obtenerYCentroCalleHorizontal devuelve la altura del centro de una calle horizontal a partir de su índice. obtenerXCentroCalleVertical calcula de la misma manera la posición horizontal de una calle vertical. Juego utiliza ambos métodos para generar enemigos sobre las calles.

Clase Casa

Casa representa uno de los posibles destinos de un pedido. Los atributos x e y indican el centro del edificio, mientras que ancho y alto determinan sus dimensiones. img contiene la imagen que se utiliza para dibujarlo. La casa conserva su posición durante toda la partida.

El constructor Casa recibe esos valores y los guarda en el objeto. Cuando dispone de una imagen, la adapta al ancho y al alto indicados mediante getScaledInstance. Esto permite que el dibujo ocupe el mismo rectángulo que se utiliza para comprobar los clics y las colisiones.

dibujar muestra la imagen en el centro de la casa. Si no hay una imagen disponible, dibuja un rectángulo de color marrón. contienePunto recibe las coordenadas de un punto y compara su distancia horizontal y vertical con la mitad de las dimensiones del edificio. Se utiliza para comprobar si el clic derecho se realizó sobre la casa que corresponde al pedido.

Clase Negocio

Negocio representa los edificios desde los que se retiran los pedidos. Sus variables x, y, ancho y alto guardan el centro y las dimensiones del edificio. img conserva su imagen. Aunque su representación es similar a la de Casa, se mantiene como una clase distinta porque cumple el papel de origen dentro de Pedido.

El constructor Negocio recibe la posición, el tamaño y la imagen, asigna los atributos y adapta el dibujo a las dimensiones del objeto. dibujar muestra esa imagen o, cuando no está disponible, un rectángulo azul. contienePunto comprueba si unas coordenadas se encuentran dentro del edificio. Juego consulta ese método para aceptar el clic de retiro únicamente sobre el negocio seleccionado.

Clase Repartidor

Repartidor contiene los datos del personaje controlado por el jugador. x e y representan su centro y cambian durante el movimiento. ancho y alto determinan su cuerpo de 20 por 20 píxeles. velocidad establece un avance de dos píxeles por actualización en cada movimiento

solicitado. dirX y dirY conservan la dirección del último desplazamiento aceptado, y img guarda la imagen del personaje.

El constructor Repartidor recibe la posición inicial y la imagen. Asigna el tamaño y la velocidad, y establece como orientación inicial la dirección hacia arriba. Después adapta la imagen a las dimensiones del personaje. Esa orientación inicial también permite definir hacia dónde sale un disparo si el jugador todavía no se movió.

colisionaCon recibe el centro y las dimensiones de otro rectángulo. Compara la distancia entre los centros en cada eje con la suma de las mitades de sus tamaños. Existe una colisión cuando ambos rectángulos se superponen horizontal y verticalmente. El contacto exacto entre sus bordes no se considera una superposición en este método.

dibujar selecciona el ángulo de la imagen a partir de dirX y dirY. La orientación cambia al avanzar hacia la derecha, la izquierda, arriba o abajo. Si la imagen no se cargó, muestra un cuadrado amarillo con las dimensiones del repartidor.

moverDerecha suma velocidad a x y corrige la posición si el borde derecho del personaje supera el límite recibido. moverIzquierda resta velocidad a x y aplica la misma comprobación sobre el borde izquierdo. moverArriba resta velocidad a y y controla el límite superior. moverAbajo suma velocidad a y y controla el límite inferior. Estos métodos se ocupan del desplazamiento y de los bordes del mapa. Juego consulta después las colisiones con la ciudad y actualiza la dirección solo si el movimiento pudo concretarse.

Clase Pedido

Pedido relaciona los edificios que participan en un encargo. origen es una referencia al Negocio donde se retira y destino es una referencia a la Casa donde debe entregarse. estado es una cadena de texto que registra la etapa actual. El objeto utiliza los edificios que ya existen en el mapa, sin crear nuevas casas o negocios.

El constructor Pedido recibe el origen y el destino y establece el estado Realizado. recibirEnNegocio cambia ese estado a Esperando. retirar permite pasar de Esperando a En camino y entregar permite pasar de En camino a Entregado. Cada método comprueba el estado previo para evitar cambios que no corresponden al recorrido normal del pedido.

perder cambia el estado a Perdido cuando el pedido todavía está Esperando o En camino. Se utiliza tanto al agotarse el plazo como cuando un enemigo roba el encargo transportado. estaActivo devuelve true para esos dos estados y false para los demás. Juego usa esa consulta al revisar el vencimiento. El control del reloj queda en Juego, mientras que Pedido conserva las referencias y las transiciones de estado.

Clase Enemigo

Enemigo representa a los personajes que recorren las calles y pueden interceptar al repartidor. Los atributos x e y indican su centro. ancho y alto establecen un tamaño de 25 por 25 píxeles, y velocidad indica un desplazamiento de 1,5 píxeles por actualización. direccionX y direccionY guardan el sentido de avance en cada eje. horizontal determina cuál de los dos ejes se utiliza.

dejaBombas indica si el enemigo puede colocar bombas. ultimaBomba registra el momento de la colocación anterior o el momento de creación del enemigo. intervaloBomba establece una espera de 4000 milisegundos entre colocaciones. Estos datos permiten que cada enemigo controle su propio intervalo.

El constructor Enemigo recibe la posición, la orientación de la calle, la capacidad de colocar bombas y el tiempo actual. Guarda esos datos, fija el tamaño y la velocidad, y establece

inicialmente el sentido positivo del recorrido. Un enemigo horizontal comienza avanzando hacia la derecha y uno vertical hacia abajo.

debeDejarBomba recibe el tiempo actual y comprueba si el enemigo tiene esa capacidad y ya completó el intervalo de espera. Cuando se cumplen ambas condiciones, actualiza ultimaBomba y devuelve true. La creación del objeto Bomba se realiza después en Juego. Actualizar el momento en esta comprobación evita que se creen varias bombas en un mismo instante.

mover recibe los cuatro límites del mapa. Si horizontal es true, modifica x y conserva y. En el otro caso modifica y y conserva x. Cuando el cuerpo alcanza un extremo, corrige la posición y cambia el sentido del recorrido para que el enemigo regrese por la misma calle.

colisionaCon utiliza la superposición de rectángulos para detectar el contacto con el repartidor. dibujar representa al enemigo con un rectángulo violeta si deja bombas y con uno rojo si no tiene esa capacidad. Esta diferencia permite reconocerlos durante la partida.

Clase Bomba

Bomba representa un objeto que permanece en el lugar donde fue colocado. x e y guardan su centro. anchoExplosion y altoExplosion definen el área que podrá causar daño cuando explote. tiempoCreacion registra el momento en que apareció y tiempoTranscurrido guarda la diferencia entre ese momento y la lectura actual del reloj. demoraExplosion establece la espera antes de explotar y duracionExplosion indica cuánto dura el efecto. explotando permite distinguir la bomba en espera de la fase de explosión.

El constructor Bomba recibe la posición, el ancho de la calle, su orientación y el tiempo actual. Para una calle horizontal crea una zona de explosión de 90 por 30 píxeles. Para una vertical utiliza 30 por 90. Estas dimensiones cubren el ancho completo de la calle y una parte de su largo. El constructor también fija una demora de 2000 milisegundos y una duración de 500 milisegundos para la explosión.

actualizar recibe el tiempo actual y calcula cuánto pasó desde la creación. Cuando ese valor alcanza la demora, establece explotando en true. terminada comprueba si ya transcurrieron tanto la demora como la duración del efecto. Juego utiliza este resultado para eliminar la referencia de la bomba del arreglo.

afectaA recibe el rectángulo del repartidor y determina si la bomba puede dañarlo. Si ya terminó, devuelve false. Mientras espera para explotar, comprueba el contacto entre el cuerpo circular de la bomba, de seis píxeles de radio, y el rectángulo recibido. Durante la explosión utiliza el rectángulo definido por anchoExplosion y altoExplosion. Esta separación permite que el daño coincida con la fase que se está mostrando.

dibujar muestra un círculo negro de 12 píxeles de diámetro mientras la bomba está en espera. Cuando explota, dibuja un rectángulo naranja con las dimensiones del área de daño. Si la bomba terminó, el método sale sin dibujarla.

Clase Proyectil

Proyectil representa el disparo del repartidor. x e y guardan su posición, ancho y alto establecen un cuerpo de 8 por 8 píxeles y velocidad determina un avance de seis píxeles por actualización. dirX y dirY conservan la dirección recibida al disparar. El proyectil mantiene esa dirección aunque el repartidor se mueva después.

El constructor Proyectil recibe la posición inicial y los componentes de dirección. Juego lo crea solamente cuando la referencia proyectil tiene valor null y se detecta un clic izquierdo. mover actualiza x e y sumando el producto de la velocidad por la dirección correspondiente.

fueraDe recibe los límites del mapa y comprueba si el rectángulo del proyectil quedó completamente afuera por alguno de sus bordes. colisionaCon determina si se superpone con otro rectángulo y se utiliza para detectar impactos contra enemigos. dibujar muestra el disparo como un rectángulo celeste. La eliminación del proyectil y el aumento del contador de enemigos eliminados se realizan en Juego después de detectar un impacto.

Problemas encontrados y decisiones tomadas

Construcción del mapa y uso de arreglos

Al principio dibujamos una cuadra y colocamos los edificios mediante coordenadas individuales. Esa primera versión ayudó a comprobar las posiciones, pero repetir el mismo código para toda la ciudad resultaba difícil de mantener. Para extender el mapa utilizamos ciclos que recorren filas y columnas y calculan el comienzo de cada cuadra.

Durante ese cambio fue necesario distinguir el número total de columnas de la columna que estaba recorriendo el ciclo. Utilizar la cantidad total al calcular el índice de una casa podía llevar a una posición fuera del arreglo. La solución consistió en obtener primero el índice de la cuadra y reservar tres lugares consecutivos para sus casas. Así cada objeto queda asociado a una posición válida y se puede recorrer el conjunto completo para dibujarlo.

También ajustamos el cálculo vertical para que utilice la altura de la cuadra. Usar el ancho en ese desplazamiento producía una separación incorrecta entre las filas. Separar los cálculos de ancho y alto permitió conservar la distribución de cuatro filas y cuatro columnas dentro del espacio disponible.

Coordenadas y tamaño de los dibujos

El entorno dibuja los rectángulos tomando como referencia su centro. En Mapa, en cambio, x e y señalan la esquina superior izquierda del área jugable. Por eso, para dibujar el fondo se suma la mitad del ancho y de la altura. En los edificios y personajes utilizamos directamente coordenadas centrales. Tener clara esta diferencia fue necesario para ubicar cada elemento y comprobar los límites del mapa.

Cuando incorporamos las imágenes apareció una diferencia entre lo que se veía y el tamaño que utilizaban los cálculos. El repartidor se dibujaba más grande que su cuerpo lógico y algunas partes visibles de los edificios quedaban fuera del área que aceptaba el clic. Ajustamos las imágenes en los constructores para que utilicen las mismas dimensiones que sus objetos. El repartidor quedó en 20 por 20 píxeles y los edificios en 50 por 30.

Después de corregir esos tamaños retiramos el margen adicional que se había sumado a las colisiones del repartidor. También incorporamos la comprobación estaEnCalle para bloquear el ingreso en las cuadras. Con esta decisión, la restricción del recorrido depende de la distribución del mapa y de las dimensiones reales del personaje.

Movimiento y dirección del disparo

Para impedir que el repartidor atraviese un obstáculo guardamos su posición antes de moverlo. Luego comprobamos el resultado y, si existe una colisión, recuperamos la posición anterior. Este procedimiento permite revisar por separado los desplazamientos horizontales y verticales.

En una versión anterior, la dirección del disparo cambiaba dentro de los métodos de movimiento. Eso hacía que, al presionar una tecla contra un obstáculo, el personaje modificara su dirección aunque permaneciera en el mismo lugar. Trasladamos esa actualización al momento en que Juego confirma que el desplazamiento se realizó. De esta forma, el disparo conserva la última dirección efectivamente recorrida.

Retiro y entrega de pedidos

Las primeras comprobaciones de retiro y entrega utilizaban la distancia al centro del edificio. Ese criterio podía aceptar acciones cuando el repartidor estaba cerca, pero fuera de la flecha indicada. Además, había más de un bloque dedicado a procesar la entrega, lo que dificultaba seguir el efecto de un mismo clic.

Reunimos ambas acciones en un solo bloque que distingue el estado del pedido. Si está Esperando, se verifica la flecha del negocio. Si está En camino, se verifica la de la casa. En los dos casos se comprueba también que el clic se haya realizado dentro del edificio correspondiente. La flecha roja del destino se muestra únicamente después del retiro.

Medición del tiempo y vencimientos

Inicialmente calculamos los segundos contando las ejecuciones de tick y suponiendo que se producían 60 por segundo. Esa frecuencia no está garantizada por el entorno y puede variar según la ejecución. El resultado era un reloj y unos plazos que dependían de la cantidad de actualizaciones realizadas.

La solución fue utilizar entorno.tiempo, que devuelve los milisegundos transcurridos desde el inicio. Para cada pedido se registra su momento de creación y se compara la diferencia con el límite de 30000 milisegundos. Las bombas y los enemigos que las colocan utilizan el mismo criterio para sus esperas. Las velocidades de desplazamiento continúan expresadas en píxeles por actualización.

También corregimos el orden en que se revisaba el vencimiento. Si se aceptaba una entrega antes de comprobar el plazo, el nuevo pedido reiniciaba el contador y podía ocultar que el anterior había llegado tarde. Ahora el vencimiento se evalúa al comienzo de tick. Cuando ocurre, se pierde el pedido y se descarta la posibilidad de usar ese mismo clic para retirar o entregar otro encargo durante esa actualización.

Daño de las bombas y eliminación de objetos

El contacto con una bomba sin explotar y el contacto con su explosión requieren comprobaciones diferentes. Aplicar el rectángulo de la explosión desde el momento de creación podía terminar la partida antes de que el jugador viera ese efecto. Por eso, afectaA distingue el círculo de la bomba en espera del rectángulo de la explosión.

También comprobamos que una explosión terminada dejara de producir daño. terminada permite consultar ese estado y Juego asigna null en la posición correspondiente del arreglo. Para agregar nuevas bombas se aprovechan primero los lugares libres. El arreglo se amplía solamente cuando todas sus posiciones siguen ocupadas por objetos activos.

La referencia del proyectil también pasa a null después de un impacto o cuando sale del mapa. Esa misma referencia se consulta antes de disparar para impedir que haya más de uno en vuelo. Cuando se elimina un enemigo, su lugar se libera y se genera el reemplazo en el mismo índice del arreglo.

Enemigos y condiciones de finalización

La generación completamente aleatoria podía dejar una partida sin enemigos capaces de colocar bombas. Para asegurar la presencia de esa mecánica, el enemigo del índice cero siempre tiene esa capacidad. Los demás pueden recibirla al azar. También evitamos que un reemplazo aparezca directamente sobre el repartidor, desplazándolo hacia un extremo de la calle cuando queda demasiado cerca.

Otra corrección consistió en separar los pedidos robados de los que vencen por tiempo. El contador general pedidosPerdidos conserva ambos casos, pero la condición de derrota por

robos utiliza pedidosRobados. Esto evita que un pedido vencido se cuente como si lo hubiera quitado un enemigo.

Para finalizar la partida utilizamos estadoJuego. Cuando toma el valor GANASTE o PERDISTE, tick deja de actualizar los movimientos y los demás objetos y se limita a mostrar el resultado. Si una derrota coincide con la entrega necesaria para ganar en una misma actualización, se evalúa primero la derrota. Esa prioridad evita que dos resultados distintos queden activos al mismo tiempo.

Revisión del funcionamiento

Durante la revisión se comprobó la compilación del proyecto y se realizaron 81 comprobaciones de lógica. Entre ellas se revisaron los clics correctos e incorrectos de retiro y entrega, los límites del mapa, las posiciones de las flechas, los vencimientos, las colisiones, los disparos, la reposición de enemigos y la duración de las bombas. Las comprobaciones finalizaron correctamente sobre la versión corregida.

Estas pruebas de lógica se ejecutaron con entradas y tiempos controlados, fuera del código entregado como parte del juego. También se revisó la representación del mapa, de los personajes y del panel con el entorno real. La combinación permitió comprobar tanto las decisiones del programa como la correspondencia entre las dimensiones de los dibujos y las que usan las interacciones.

Implementación

La implementación está escrita en Java y sus nueve clases pertenecen al paquete juego. El proyecto utiliza entorno.jar para la ventana, el dibujo y la lectura de entradas. Los archivos casa_uno.png, casa_dos.png y casa_tres.png contienen las imágenes de las casas. negocio.png y repartidor.png corresponden al negocio y al personaje. Las imágenes se cargan desde la carpeta de ejecución del proyecto, por lo que se conservan junto a los archivos principales del repositorio.

El programa comienza en el método main de Juego. Para ejecutarlo se utiliza el proyecto con su biblioteca entorno.jar y las imágenes disponibles en la carpeta de trabajo. La configuración de Eclipse está incluida en .project y .classpath. Git se utilizó para registrar los cambios del desarrollo y conservar las versiones compartidas del código.

El código que se presenta a continuación corresponde a las nueve clases implementadas. Se mantiene su funcionamiento y sus comentarios, con la sangría y los saltos de línea acomodados para facilitar la lectura en el documento. La biblioteca del entorno forma parte del material proporcionado por la materia y no se reproduce como código propio.

Código de Juego

```
package juego;

import java.awt.Color;
import entorno.Entorno;
import entorno.InterfaceJuego;

public class Juego extends InterfaceJuego {
    Entorno entorno;

    Mapa mapa;
    Repartidor repartidor;
    Pedido pedido;

    Enemigo[] enemigos;
    Bomba[] bombas;
    Proyectil proyectil;

    int pedidosEntregados;
    int pedidosPerdidos;
    int pedidosRobados;
```

```
int enemigosEliminados;
int vidas;
long tiempoActual;
long inicioPedido;
int tiempoLimitePedido;

int pedidosParaGanar;
int pedidosRobadosParaPerder;
int vidasIniciales;

String estadoJuego;

Juego() {
    this.entorno = new Entorno(this, "Delivery Rush", 800, 600);
    this.mapa = new Mapa(0, 60, 800, 540);

    // Cargar la imagen; si no se encuentra, se dibuja un rectángulo.
    java.awt.Image imgRepartidor = null;
    try {
        imgRepartidor = javax.imageio.ImageIO.read(new java.io.File("repartidor.png"));
    } catch (Exception e) {
        System.out.println("No se pudo cargar repartidor.png");
    }

    // Pasamos la imagen (si es null, dibuja el cuadrado amarillo)
    this.repartidor = new Repartidor(400, 585, imgRepartidor);

    this.vidasIniciales = 3;
    this.vidas = this.vidasIniciales;
    this.pedidosParaGanar = 5;
    this.pedidosRobadosParaPerder = 5;
    this.tiempoLimitePedido = 30000; // Milisegundos: 30 segundos.

    this.pedidosEntregados = 0;
    this.pedidosPerdidos = 0;
    this.pedidosRobados = 0;
    this.enemigosEliminados = 0;
    this.tiempoActual = 0;
    this.inicioPedido = 0;

    this.bombas = new Bomba[0];
    this.proyectil = null;

    this.enemigos = new Enemigo[4];
    this.generarEnemigo(0);
    this.generarEnemigo(1);
    this.generarEnemigo(2);
    this.generarEnemigo(3);

    this.crearNuevoPedido();
    this.estadoJuego = "Jugando";
    this.entorno.iniciar();
}

void generarEnemigo(int indice) {
    boolean horizontal = Math.random() < 0.5;
    // El lugar cero garantiza al menos un enemigo que deja bombas.
    boolean dejaBombas = indice == 0 || Math.random() < 0.4;
    double x, y;
    if (horizontal) {
        int filaCalle = (int) (Math.random() * (this.mapa.filas + 1));
        y = this.mapa.obtenerYCentroCalleHorizontal(filaCalle);
        x = this.mapa.x + 20 + Math.random() * (this.mapa.ancho - 40);
    } else {
        int colCalle = (int) (Math.random() * (this.mapa.columnas + 1));
        x = this.mapa.obtenerXCentroCalleVertical(colCalle);
        y = this.mapa.y + 20 + Math.random() * (this.mapa.alto - 40);
    }
    // Evitar que un reemplazo aparezca encima del repartidor.
    if (Math.abs(x - this.repartidor.x) < 80 && Math.abs(y - this.repartidor.y) < 80) {
        if (horizontal) {
            if (this.repartidor.x < this.mapa.x + this.mapa.ancho / 2) {
                x = this.mapa.x + this.mapa.ancho - 20;
            } else {
                x = this.mapa.x + 20;
            }
        } else {
            if (this.repartidor.y < this.mapa.y + this.mapa.alto / 2) {
                y = this.mapa.y + this.mapa.alto - 20;
            } else {
                y = this.mapa.y + 20;
            }
        }
    }
```

```
    this.enemigos[indice] = new Enemigo(x, y, horizontal, dejaBombas, this.tiempoActual);
}

void crearNuevoPedido() {
    int indiceNegocio = (int) (Math.random() * this.mapa.negocios.length);
    int indiceCasa = (int) (Math.random() * this.mapa.casas.length);
    this.pedido = new Pedido(this.mapa.negocios[indiceNegocio],
        this.mapa.casas[indiceCasa]);
    this.pedido.recibirEnNegocio();
    this.inicioPedido = this.tiempoActual;
}

public void tick() {

    if (!this.estadoJuego.equals("Jugando")) {
        this.dibujarPantallaFinal(this.estadoJuego);
        return;
    }

    // El reloj del entorno mide milisegundos, sin suponer una cantidad de ticks.
    this.tiempoActual = this.entorno.tiempo();
    boolean pedidoVencido = this.pedido.estaActivo()
        && this.tiempoActual - this.inicioPedido >= this.tiempoLimitePedido;
    if (pedidoVencido) {
        this.pedido.perder();
        this.pedidosPerdidos++;
        this.crearNuevoPedido();
    }

    // Movimiento del repartidor
    if (this.entorno.estaPresionada('d') ||
        this.entorno.estaPresionada(this.entorno.TECLA_DERECHA)) {
        double xAnterior = this.repartidor.x;
        this.repartidor.moverDerecha(this.mapa.x + this.mapa.ancho);
        if (this.mapa.hayColision(this.repartidor)) {
            this.repartidor.x = xAnterior;
        } else if (this.repartidor.x != xAnterior) {
            this.repartidor.dirX = 1;
            this.repartidor.dirY = 0;
        }
    }
    if (this.entorno.estaPresionada('a') ||
        this.entorno.estaPresionada(this.entorno.TECLA_IZQUIERDA)) {
        double xAnterior = this.repartidor.x;
        this.repartidor.moverIzquierda(this.mapa.x);
        if (this.mapa.hayColision(this.repartidor)) {
            this.repartidor.x = xAnterior;
        } else if (this.repartidor.x != xAnterior) {
            this.repartidor.dirX = -1;
            this.repartidor.dirY = 0;
        }
    }
    if (this.entorno.estaPresionada('w') ||
        this.entorno.estaPresionada(this.entorno.TECLA_ARRIBA)) {
        double yAnterior = this.repartidor.y;
        this.repartidor.moverArriba(this.mapa.y);
        if (this.mapa.hayColision(this.repartidor)) {
            this.repartidor.y = yAnterior;
        } else if (this.repartidor.y != yAnterior) {
            this.repartidor.dirX = 0;
            this.repartidor.dirY = -1;
        }
    }
    if (this.entorno.estaPresionada('s') ||
        this.entorno.estaPresionada(this.entorno.TECLA_ABAJO)) {
        double yAnterior = this.repartidor.y;
        this.repartidor.moverAbajo(this.mapa.y + this.mapa.alto);
        if (this.mapa.hayColision(this.repartidor)) {
            this.repartidor.y = yAnterior;
        } else if (this.repartidor.y != yAnterior) {
            this.repartidor.dirX = 0;
            this.repartidor.dirY = 1;
        }
    }

    // Retiro y entrega: estar sobre la flecha y hacer clic en el edificio.
    if (!pedidoVencido && this.entorno.sePresionoBoton(this.entorno.BOTON_DERECHO)) {
        if (this.pedido.estado.equals("Esperando")) {
            Negocio negocio = this.pedido.origen;
            double yFlecha = this.mapa.obtenerYFlechaNegocio(negocio);

            boolean sobreFlecha =
                Math.abs(this.repartidor.x - negocio.x) <= this.repartidor.ancho / 2
                && Math.abs(this.repartidor.y - yFlecha) <= this.repartidor.alto / 2;
```

```
            boolean clicCorrecto = negocio.contienePunto(
                this.entorno.mouseX(), this.entorno.mouseY()
                );

            if (sobreFlecha && clicCorrecto) {
                this.pedido.retirar();
            }
        } else if (this.pedido.estado.equals("En camino")) {
            Casa casa = this.pedido.destino;
            double yFlecha = this.mapa.obtenerYFlechaCasa(casa);

            boolean sobreFlecha =
                Math.abs(this.repartidor.x - casa.x) <= this.repartidor.ancho / 2
                && Math.abs(this.repartidor.y - yFlecha) <= this.repartidor.alto / 2;

            boolean clicCorrecto = casa.contienePunto(
                this.entorno.mouseX(), this.entorno.mouseY()
                );

            if (sobreFlecha && clicCorrecto) {
                this.pedido.entregar();
                this.pedidosEntregados++;
                this.crearNuevoPedido();
            }
        }
    }

    // Disparo
    if (this.entorno.sePresionoBoton(this.entorno.BOTON_IZQUIERDO) && this.proyectil ==
        null) {
        this.proyectil = new Proyectil(this.repartidor.x, this.repartidor.y,
            this.repartidor.dirX, this.repartidor.dirY);
    }

    // Actualizar proyectil
    if (this.proyectil != null) {
        this.proyectil.mover();
        for (int i = 0; i < this.enemigos.length; i++) {
            Enemigo e = this.enemigos[i];
            if (e != null && this.proyectil != null && this.proyectil.colisionaCon(e.x, e.y,
                e.ancho, e.alto)) {
                this.enemigos[i] = null;
                this.proyectil = null;
                this.enemigosEliminados++;
                this.generarEnemigo(i);
                break;
            }
        }
        if (this.proyectil != null && this.proyectil.fueraDe(this.mapa.x, this.mapa.x +
            this.mapa.ancho, this.mapa.y, this.mapa.y + this.mapa.alto)) {
            this.proyectil = null;
        }
    }

    // Actualizar enemigos
    for (int i = 0; i < this.enemigos.length; i++) {
        Enemigo e = this.enemigos[i];
        if (e == null) continue;
        e.mover(this.mapa.x, this.mapa.x + this.mapa.ancho, this.mapa.y, this.mapa.y +
            this.mapa.alto);
        if (e.debeDejarBomba(this.tiempoActual)) {
            this.agregarBomba(new Bomba(
                e.x, e.y, this.mapa.anchoCalle, e.horizontal, this.tiempoActual
                ));
        }
        if (e.colisionaCon(this.repartidor.x, this.repartidor.y, this.repartidor.ancho,
            this.repartidor.alto)) {
            this.vidas--;
            if (this.pedido.estado.equals("En camino")) {
                this.pedido.perder();
                this.pedidosPerdidos++;
                this.pedidosRobados++;
                this.crearNuevoPedido();
            }
            this.enemigos[i] = null;
            this.generarEnemigo(i);
        }
    }

    // Actualizar bombas
    for (int i = 0; i < this.bombas.length; i++) {
        Bomba b = this.bombas[i];
        if (b == null) continue;
        b.actualizar(this.tiempoActual);
        if (b.afectaA(this.repartidor.x, this.repartidor.y, this.repartidor.ancho,
```

```
            this.repartidor.alto)) {
            this.vidas = 0;
        }
        if (b.terminada()) this.bombas[i] = null;
    }

    // Una derrota tiene prioridad si ocurre al mismo tiempo que la última entrega.
    if (this.vidas <= 0 || this.pedidosRobados >= this.pedidosRobadosParaPerder) {
        this.estadoJuego = "PERDISTE";
    } else if (this.pedidosEntregados >= this.pedidosParaGanar) {
        this.estadoJuego = "GANASTE";
    }

    if (!this.estadoJuego.equals("Jugando")) {
        this.dibujarPantallaFinal(this.estadoJuego);
        return;
    }

    // ---- DIBUJAR ----
    this.mapa.dibujar(this.entorno);

    // Dibujar flechas
    if (this.pedido.estado.equals("En camino")) {
        this.mapa.dibujarFlechaCasaRoja(this.entorno, this.pedido.destino);
    }
    if (this.pedido.estado.equals("Esperando")) {
        this.mapa.dibujarFlechaNegocio(this.entorno, this.pedido.origen);
    }

    for (int i = 0; i < this.enemigos.length; i++) {
        if (this.enemigos[i] != null) this.enemigos[i].dibujar(this.entorno);
    }
    for (int i = 0; i < this.bombas.length; i++) {
        if (this.bombas[i] != null) this.bombas[i].dibujar(this.entorno);
    }
    if (this.proyectil != null) this.proyectil.dibujar(this.entorno);

    this.repartidor.dibujar(this.entorno);
    this.dibujarPanel();
}

void agregarBomba(Bomba b) {
    // Reutilizar los lugares de bombas que ya terminaron.
    for (int i = 0; i < this.bombas.length; i++) {
        if (this.bombas[i] == null || this.bombas[i].terminada()) {
            this.bombas[i] = null;
            this.bombas[i] = b;
            return;
        }
    }
    Bomba[] nuevo = new Bomba[this.bombas.length + 1];
    for (int i = 0; i < this.bombas.length; i++) nuevo[i] = this.bombas[i];
    nuevo[this.bombas.length] = b;
    this.bombas = nuevo;
}

void dibujarPanel() {
    this.entorno.dibujarRectangulo(400, 30, 800, 60, 0, new Color(30, 30, 30));
    this.entorno.cambiarFont("Arial", 14, Color.WHITE);

    String tiempo = "Tiempo: " + (this.tiempoActual / 1000) + "s";
    String vidasStr = "Vidas: " + this.vidas;
    String entregados = "Entregados: " + this.pedidosEntregados;
    String eliminados = "Eliminados: " + this.enemigosEliminados;
    String estado = "Pedido: " + this.pedido.estado;

    long restante = this.tiempoLimitePedido - (this.tiempoActual - this.inicioPedido);
    long segundosRestantes = Math.max(0, (restante + 999) / 1000);

    this.entorno.escribirTexto(tiempo, 15, 24);
    this.entorno.escribirTexto(vidasStr, 160, 24);
    this.entorno.escribirTexto(entregados, 280, 24);
    this.entorno.escribirTexto(eliminados, 460, 24);
    this.entorno.escribirTexto(estado, 15, 48);
    this.entorno.escribirTexto("Plazo: " + segundosRestantes + "s", 280, 48);
    this.entorno.escribirTexto("Robados: " + this.pedidosRobados, 460, 48);
}

void dibujarPantallaFinal(String mensaje) {
    this.entorno.dibujarRectangulo(400, 300, 800, 600, 0, Color.BLACK);
    this.entorno.cambiarFont("Arial", 48, Color.WHITE);
    this.entorno.escribirTexto(mensaje, 300, 300);
}

@SuppressWarnings("unused")
```

```
    public static void main(String[] args) {
        Juego juego = new Juego();
    }
}
```

Código de Mapa

```
package juego;

import java.awt.Color;
import java.awt.Image;
import entorno.Entorno;

public class Mapa {
    double x;
    double y;
    double ancho;
    double alto;

    int filas;
    int columnas;
    int anchoCalle;

    Casa[] casas;
    Negocio[] negocios;

    public Mapa(double x, double y, double ancho, double alto){
        this.x = x;
        this.y = y;
        this.ancho = ancho;
        this.alto = alto;
        this.filas = 4;
        this.columnas = 4;
        this.anchoCalle = 30;

        // CARGAR LAS DISTINTAS CASAS Y NEGOCIOS
        Image[] imagenesCasas = new Image[3];
        Image imgNegocio = null;
        try {
            imagenesCasas[0] = javax.imageio.ImageIO.read(new java.io.File("casa_dos.png"));
            imagenesCasas[1] = javax.imageio.ImageIO.read(new java.io.File("casa_uno.png"));
            imagenesCasas[2] = javax.imageio.ImageIO.read(new java.io.File("casa_tres.png"));
            imgNegocio = javax.imageio.ImageIO.read(new java.io.File("negocio.png"));
        } catch (Exception e) {
            System.out.println("No se pudieron cargar las imágenes.");
        }
        // ---------------------------------------------

        this.casas = new Casa[this.filas * this.columnas * 3];
        this.negocios = new Negocio[this.filas * this.columnas];

        double anchoCuadra = (this.ancho - (this.columnas +1) * this.anchoCalle) /
            this.columnas;
        double altoCuadra = (this.alto - (this.filas + 1) * this.anchoCalle) / this.filas;

        for (int fila = 0; fila < this.filas; fila++) {
            double inicioY = this.y + this.anchoCalle + fila * (altoCuadra + this.anchoCalle);
            for (int columna = 0; columna < this.columnas; columna++) {
                double inicioX = this.x + this.anchoCalle + columna * (anchoCuadra +
                    this.anchoCalle);
                int indiceNegocio = fila * this.columnas + columna;
                int indiceCasa = indiceNegocio * 3;

                int azar1 = (int)(Math.random() * imagenesCasas.length);
                int azar2 = (int)(Math.random() * imagenesCasas.length);
                int azar3 = (int)(Math.random() * imagenesCasas.length);

                this.casas[indiceCasa] = new Casa(inicioX + 40, inicioY + 25, 50, 30,
                    imagenesCasas[azar1]);
                this.casas[indiceCasa + 1] = new Casa(inicioX + 120, inicioY + 25, 50, 30,
                    imagenesCasas[azar2]);
                this.casas[indiceCasa + 2] = new Casa(inicioX + 40, inicioY + 70, 50, 30,
                    imagenesCasas[azar3]);

                this.negocios[indiceNegocio] = new Negocio(inicioX + 120, inicioY + 70, 50, 30,
                    imgNegocio);
            }
        }
    }
    // El repartidor debe permanecer por completo sobre las calles.
    public boolean estaEnCalle(Repartidor repartidor) {
        if (repartidor.x - repartidor.ancho / 2 < this.x
            || repartidor.x + repartidor.ancho / 2 > this.x + this.ancho
            || repartidor.y - repartidor.alto / 2 < this.y
```

```
            || repartidor.y + repartidor.alto / 2 > this.y + this.alto) {
            return false;
        }
        double anchoCuadra = (this.ancho - (this.columnas + 1) * this.anchoCalle) /
            this.columnas;
        double altoCuadra = (this.alto - (this.filas + 1) * this.anchoCalle) / this.filas;
        for (int fila = 0; fila < this.filas; fila++) {
            double centroY = this.y + this.anchoCalle + fila * (altoCuadra + this.anchoCalle) +
                altoCuadra / 2;
            for (int columna = 0; columna < this.columnas; columna++) {
                double centroX = this.x + this.anchoCalle + columna * (anchoCuadra +
                    this.anchoCalle) + anchoCuadra / 2;
                if (repartidor.colisionaCon(centroX, centroY, anchoCuadra, altoCuadra)) {
                    return false;
                }
            }
        }
        return true;
    }

    public boolean hayColision(Repartidor repartidor) {
        if (!this.estaEnCalle(repartidor)) {
            return true;
        }
        for (int i = 0; i < this.casas.length; i++) {
            Casa casa = this.casas[i];
            if (repartidor.colisionaCon(casa.x, casa.y, casa.ancho, casa.alto)) return true;
        }
        for (int i = 0; i < this.negocios.length; i++) {
            Negocio negocio = this.negocios[i];
            if (repartidor.colisionaCon(negocio.x, negocio.y, negocio.ancho, negocio.alto))
                return true;
        }
        return false;
    }

    public double obtenerYFlechaCasa(Casa casa) {
        double altoCuadra = (this.alto - (this.filas + 1) * this.anchoCalle) / this.filas;
        int fila = (int) ((casa.y - this.y) / (altoCuadra + this.anchoCalle));
        double inicioY = this.y + this.anchoCalle + fila * (altoCuadra + this.anchoCalle);
        double centroCuadraY = inicioY + altoCuadra / 2;
        if (casa.y < centroCuadraY) {
            return inicioY - this.anchoCalle / 2.0;
        } else {
            return inicioY + altoCuadra + this.anchoCalle / 2.0;
        }
    }

    public void dibujarFlechaCasaRoja(Entorno entorno, Casa casa) {
        double xFlecha = casa.x;
        double yFlecha = this.obtenerYFlechaCasa(casa);
        int direccion = 1;
        if (casa.y < yFlecha) direccion = -1;
        entorno.dibujarRectangulo(xFlecha, yFlecha - direccion * 5, 6, 8, 0, Color.RED);
        entorno.dibujarTriangulo(xFlecha, yFlecha + direccion * 3, 12, 18, direccion * Math.PI /
            2, Color.RED);
    }

    public double obtenerYFlechaNegocio(Negocio negocio) {
        double altoCuadra = (this.alto - (this.filas + 1) * this.anchoCalle) / this.filas;
        int fila = (int) ((negocio.y - this.y) / (altoCuadra + this.anchoCalle));
        double bordeSuperiorCalleAbajo = this.y + this.anchoCalle + fila * (altoCuadra +
            this.anchoCalle) + altoCuadra;
        return bordeSuperiorCalleAbajo + this.anchoCalle / 2.0;
    }

    public void dibujarFlechaNegocio(Entorno entorno, Negocio negocio) {
        double xFlecha = negocio.x;
        double yFlecha = this.obtenerYFlechaNegocio(negocio);
        entorno.dibujarRectangulo(xFlecha, yFlecha + 5, 6, 8, 0, Color.YELLOW);
        entorno.dibujarTriangulo(xFlecha, yFlecha - 3, 12, 18, -Math.PI / 2, Color.YELLOW);
    }

    public void dibujar(Entorno entorno){
        entorno.dibujarRectangulo(this.x + this.ancho/2, this.y + this.alto/2, this.ancho,
            this.alto, 0, new Color(70, 70, 70));
        double anchoCuadra = (this.ancho - (this.columnas +1) * this.anchoCalle) /
            this.columnas;
        double altoCuadra = (this.alto - (this.filas + 1) * this.anchoCalle) / this.filas;

        for(int fila =0; fila < this.filas; fila ++){
            double centroY = this.y + this.anchoCalle + fila *(altoCuadra + this.anchoCalle) +
                altoCuadra /2;
            for(int columna =0; columna < this.columnas ; columna ++ ){
                double centroX = this.x + this.anchoCalle + columna * (anchoCuadra +
```

```
                    this.anchoCalle) + anchoCuadra /2;
                entorno.dibujarRectangulo(centroX, centroY, anchoCuadra, altoCuadra, 0, new
                    Color(90, 140, 80));
            }
        }
        for(int i = 0; i< this.casas.length; i++) this.casas[i].dibujar(entorno);
        for(int i =0; i< this.negocios.length; i++) this.negocios[i].dibujar(entorno);
    }

    public double obtenerYCentroCalleHorizontal(int indiceCalle) {
        double altoCuadra = (this.alto - (this.filas + 1) * this.anchoCalle) / this.filas;
        return this.y + indiceCalle * (altoCuadra + this.anchoCalle) + this.anchoCalle / 2.0;
    }

    public double obtenerXCentroCalleVertical(int indiceCalle) {
        double anchoCuadra = (this.ancho - (this.columnas + 1) * this.anchoCalle) /
            this.columnas;
        return this.x + indiceCalle * (anchoCuadra + this.anchoCalle) + this.anchoCalle / 2.0;
    }
}
```

Código de Casa

```
package juego;

import java.awt.Color;
import java.awt.Image;
import entorno.Entorno;

public class Casa {
    double x;
    double y;
    double ancho;
    double alto;
    Image img;

    public Casa(double x, double y, double ancho, double alto, Image img){
        this.x = x;
        this.y = y;
        this.ancho = ancho;
        this.alto = alto;
        this.img = img;
        if (img != null) {
            this.img = img.getScaledInstance((int) this.ancho, (int) this.alto,
                Image.SCALE_SMOOTH);
        }
    }

    public void dibujar(Entorno entorno){
        if (this.img != null) {
            entorno.dibujarImagen(this.img, this.x, this.y, 0, 1);
        } else {
            entorno.dibujarRectangulo(this.x, this.y, this.ancho, this.alto, 0, new Color(190,
                110, 80));
        }
    }

    public boolean contienePunto(double puntoX, double puntoY) {
        return Math.abs(this.x - puntoX) <= this.ancho / 2 && Math.abs(this.y - puntoY) <=
            this.alto / 2;
    }
}
```

Código de Negocio

```
package juego;

import java.awt.Color;
import java.awt.Image;
import entorno.Entorno;

public class Negocio {
    double x;
    double y;
    double ancho;
    double alto;
    Image img;
```

```
    public Negocio(double x, double y, double ancho, double alto, Image img){
        this.x = x;
        this.y = y;
        this.ancho = ancho;
        this.alto = alto;
        this.img = img;
        if (img != null) {
            this.img = img.getScaledInstance((int) this.ancho, (int) this.alto,
                Image.SCALE_SMOOTH);
        }
    }

    public void dibujar(Entorno entorno){
        if (this.img != null) {
            entorno.dibujarImagen(this.img, this.x, this.y, 0, 1);
        } else {
            entorno.dibujarRectangulo(this.x, this.y, this.ancho, this.alto, 0, new Color(70,
                130, 200));
        }
    }

    public boolean contienePunto(double puntoX, double puntoY) {
        return Math.abs(this.x - puntoX) <= this.ancho / 2 && Math.abs(this.y - puntoY) <=
            this.alto / 2;
    }
}
```

Código de Repartidor

```
package juego;

import java.awt.Color;
import entorno.Entorno;
import java.awt.Image;

public class Repartidor {
    double x;
    double y;
    double ancho;
    double alto;
    double velocidad;
    double dirX;
    double dirY;

    Image img;

    public Repartidor(double x, double y, Image img){
        this.x = x;
        this.y = y;
        this.ancho = 20;
        this.alto = 20;
        this.velocidad = 2;
        this.dirX = 0;
        this.dirY = -1;
        this.img = img;
        if (img != null) {
            this.img = img.getScaledInstance((int) this.ancho, (int) this.alto,
                Image.SCALE_SMOOTH);
        }
    }

    public boolean colisionaCon(double otroX, double otroY, double otroAncho, double otroAlto) {
        boolean coincideX = Math.abs(this.x - otroX) < (this.ancho + otroAncho) / 2;
        boolean coincideY = Math.abs(this.y - otroY) < (this.alto + otroAlto) / 2;
        return coincideX && coincideY;
    }

    // Método para dibujar al repartidor
    public void dibujar(Entorno entorno){
        if (this.img != null) {
            double angulo = 0;

            if (this.dirX == 1) {
                angulo = 0; //
            } else if (this.dirX == -1) {
                angulo = Math.PI;
            } else if (this.dirY == -1) {
                angulo = -Math.PI / 2;
            } else if (this.dirY == 1) {
                angulo = Math.PI / 2; }

            entorno.dibujarImagen(this.img, this.x, this.y, angulo, 1);
```

```
        } else {
            entorno.dibujarRectangulo(this.x, this.y, this.ancho, this.alto, 0, Color.YELLOW);
        }
    }

    public void moverDerecha(double limiteDerecho) {
        this.x = this.x + this.velocidad;
        if (this.x + this.ancho / 2 > limiteDerecho) {
            this.x = limiteDerecho - this.ancho / 2;
        }
    }

    public void moverIzquierda(double limiteIzquierdo) {
        this.x = this.x - this.velocidad;
        if (this.x - this.ancho / 2 < limiteIzquierdo) {
            this.x = limiteIzquierdo + this.ancho / 2;
        }
    }

    public void moverArriba(double limiteSuperior) {
        this.y = this.y - this.velocidad;
        if (this.y - this.alto / 2 < limiteSuperior) {
            this.y = limiteSuperior + this.alto / 2;
        }
    }

    public void moverAbajo(double limiteInferior) {
        this.y = this.y + this.velocidad;
        if (this.y + this.alto / 2 > limiteInferior) {
            this.y = limiteInferior - this.alto / 2;
        }
    }
}
```

Código de Pedido

```
package juego;

public class Pedido {
    Negocio origen;
    Casa destino;
    String estado;

    public Pedido(Negocio origen, Casa destino){
        this.origen = origen;
        this.destino = destino;
        this.estado = "Realizado";
    }

    public void recibirEnNegocio(){
        if(this.estado.equals("Realizado")){
            this.estado = "Esperando";
        }
    }

    public void retirar() {
        if (this.estado.equals("Esperando")) {
            this.estado = "En camino";
        }
    }

    public void entregar() {
        if (this.estado.equals("En camino")) {
            this.estado = "Entregado";
        }
    }

    public void perder() {
        if (this.estado.equals("En camino") || this.estado.equals("Esperando")) {
            this.estado = "Perdido";
        }
    }

    public boolean estaActivo() {
        return this.estado.equals("Esperando") || this.estado.equals("En camino");
    }
}
```

Código de Enemigo

```
package juego;

import java.awt.Color;
import entorno.Entorno;

public class Enemigo {
    double x;
    double y;
    double ancho;
    double alto;
    double velocidad;

    // Dirección: 1 = derecha/abajo, -1 = izquierda/arriba
    int direccionX;
    int direccionY;

    // true si se mueve horizontalmente, false si es vertical
    boolean horizontal;

    // Puede dejar bombas o no
    boolean dejaBombas;

    // Todos los tiempos se expresan en milisegundos.
    long ultimaBomba;
    int intervaloBomba;

    public Enemigo(double x, double y, boolean horizontal, boolean dejaBombas, long
        tiempoActual) {
        this.x = x;
        this.y = y;
        this.ancho = 25;
        this.alto = 25;
        this.velocidad = 1.5;
        this.horizontal = horizontal;
        this.dejaBombas = dejaBombas;
        this.direccionX = 1;
        this.direccionY = 1;
        this.intervaloBomba = 4000;
        this.ultimaBomba = tiempoActual;
    }

    // Devuelve true una sola vez por intervalo y registra el momento de la bomba.
    public boolean debeDejarBomba(long tiempoActual) {
        if (this.dejaBombas && tiempoActual - this.ultimaBomba >= this.intervaloBomba) {
            this.ultimaBomba = tiempoActual;
            return true;
        }
        return false;
    }

    public void dibujar(Entorno entorno) {
        Color color = this.dejaBombas ? new Color(150, 0, 200) : new Color(200, 40, 40);
        entorno.dibujarRectangulo(this.x, this.y, this.ancho, this.alto, 0, color);
    }

    // Mueve al enemigo dentro de los límites dados. Si toca un borde, cambia de dirección.
    public void mover(double limiteIzq, double limiteDer, double limiteSup, double limiteInf) {
        if (this.horizontal) {
            this.x += this.velocidad * this.direccionX;
            if (this.x + this.ancho / 2 > limiteDer) {
                this.x = limiteDer - this.ancho / 2;
                this.direccionX = -1;
            } else if (this.x - this.ancho / 2 < limiteIzq) {
                this.x = limiteIzq + this.ancho / 2;
                this.direccionX = 1;
            }
        } else {
            this.y += this.velocidad * this.direccionY;
            if (this.y + this.alto / 2 > limiteInf) {
                this.y = limiteInf - this.alto / 2;
                this.direccionY = -1;
            } else if (this.y - this.alto / 2 < limiteSup) {
                this.y = limiteSup + this.alto / 2;
                this.direccionY = 1;
            }
        }
    }
```

```
    public boolean colisionaCon(double otroX, double otroY, double otroAncho, double otroAlto) {
        boolean coincideX = Math.abs(this.x - otroX) < (this.ancho + otroAncho) / 2;
        boolean coincideY = Math.abs(this.y - otroY) < (this.alto + otroAlto) / 2;
        return coincideX && coincideY;
    }
}
```

Código de Bomba

```
package juego;

import java.awt.Color;
import entorno.Entorno;

public class Bomba {
    double x;
    double y;
    double anchoExplosion;
    double altoExplosion;

    long tiempoCreacion;
    long tiempoTranscurrido;
    int demoraExplosion;
    int duracionExplosion;
    boolean explotando;

    public Bomba(double x, double y, double anchoCalle, boolean esHorizontal, long tiempoActual)
        {
        this.x = x;
        this.y = y;

        if (esHorizontal) {
            this.anchoExplosion = anchoCalle * 3; // Largo de la calle
            this.altoExplosion = anchoCalle;      // Ancho de la calle
        } else {
            this.anchoExplosion = anchoCalle;      // Ancho de la calle
            this.altoExplosion = anchoCalle * 3;   // Largo de la calle
        }

        this.tiempoCreacion = tiempoActual;
        this.tiempoTranscurrido = 0;
        this.demoraExplosion = 2000;
        this.duracionExplosion = 500;
        this.explotando = false;
    }

    public void actualizar(long tiempoActual) {
        this.tiempoTranscurrido = tiempoActual - this.tiempoCreacion;
        this.explotando = this.tiempoTranscurrido >= this.demoraExplosion;
    }

    public boolean terminada() {
        return this.tiempoTranscurrido >= this.demoraExplosion + this.duracionExplosion;
    }

    public boolean afectaA(double otroX, double otroY, double otroAncho, double otroAlto) {
        if (this.terminada()) {
            return false;
        }

        if (!this.explotando) {
            double distanciaX = Math.max(Math.abs(this.x - otroX) - otroAncho / 2, 0);

            double distanciaY = Math.max(Math.abs(this.y - otroY) - otroAlto / 2, 0);

            return distanciaX * distanciaX + distanciaY * distanciaY <= 36;
        }

        boolean coincideX =Math.abs(this.x - otroX) < (this.anchoExplosion + otroAncho) / 2;

        boolean coincideY =Math.abs(this.y - otroY) < (this.altoExplosion + otroAlto) / 2;

        return coincideX && coincideY;
    }
```

```
    public void dibujar(Entorno entorno) {
        if (this.terminada()) {
            return;
        }

        if (this.explotando) {
            entorno.dibujarRectangulo(this.x, this.y,this.anchoExplosion, this.altoExplosion,0,
                new Color(255, 120, 0));
        }
        else {
            entorno.dibujarCirculo(this.x, this.y, 12, Color.BLACK);
        }
    }
}
```

Código de Proyectil

```
package juego;

import java.awt.Color;
import entorno.Entorno;

public class Proyectil {
    double x;
    double y;
    double ancho;
    double alto;
    double velocidad;
    double dirX;
    double dirY;

    public Proyectil(double x, double y, double dirX, double dirY) {
        this.x = x;
        this.y = y;
        this.ancho = 8;
        this.alto = 8;
        this.velocidad = 6;
        this.dirX = dirX;
        this.dirY = dirY;
    }

    public void mover() {
        this.x += this.dirX * this.velocidad;
        this.y += this.dirY * this.velocidad;
    }

    public boolean fueraDe(double limiteIzq, double limiteDer, double limiteSup, double
        limiteInf) {
        return this.x + this.ancho / 2 < limiteIzq
            || this.x - this.ancho / 2 > limiteDer
            || this.y + this.alto / 2 < limiteSup
            || this.y - this.alto / 2 > limiteInf;
    }

    public boolean colisionaCon(double otroX, double otroY, double otroAncho, double otroAlto) {
        boolean coincideX = Math.abs(this.x - otroX) < (this.ancho + otroAncho) / 2;
        boolean coincideY = Math.abs(this.y - otroY) < (this.alto + otroAlto) / 2;
        return coincideX && coincideY;
    }

    public void dibujar(Entorno entorno) {
        entorno.dibujarRectangulo(this.x, this.y, this.ancho, this.alto, 0, Color.CYAN);
    }
}
```

Conclusiones

El desarrollo de Delivery Rush permitió reunir en una misma aplicación los conceptos de objetos, métodos, arreglos y control de flujo trabajados durante el proyecto. El juego cuenta con una ciudad organizada en cuadras, un repartidor que se mueve por las calles, pedidos con origen y destino, enemigos, bombas y un sistema de disparo. Las entregas y los encuentros con los enemigos modifican el estado de la partida hasta llegar a una victoria o una derrota.

Uno de los aprendizajes más concretos fue la importancia de relacionar correctamente la representación gráfica con los datos del programa. Las coordenadas, las dimensiones de las imágenes y las zonas de colisión tienen que responder al mismo criterio. Cuando esas medidas no coincidían, una acción que parecía válida en pantalla podía ser rechazada por el código.

El trabajo con pedidos y temporizadores también mostró que el orden de las comprobaciones cambia el resultado. Revisar el vencimiento antes de procesar una entrega y conservar la dirección del último movimiento aceptado fueron decisiones pequeñas que corrigieron comportamientos visibles durante la partida. Para resolverlas fue necesario seguir qué valores cambiaban dentro de tick y en qué momento ocurría cada modificación.

La separación en clases ayudó a localizar esas responsabilidades. Pedido concentra los cambios de estado, Mapa calcula las posiciones y las restricciones de circulación, y los personajes y ataques conservan sus propios datos. Juego reúne esas partes para aplicar las reglas generales. Esta organización facilita relacionar cada comportamiento del juego con el fragmento de código que lo produce.

La versión final incorpora las mecánicas obligatorias y las correcciones surgidas de la revisión. El desarrollo también dejó una forma de trabajo útil para proyectos posteriores, avanzar con partes que puedan comprobarse, revisar cómo se relacionan y conservar cambios pequeños en el repositorio. Las pruebas de los límites y de las situaciones de error resultaron tan necesarias como comprobar que un recorrido normal pudiera completarse.